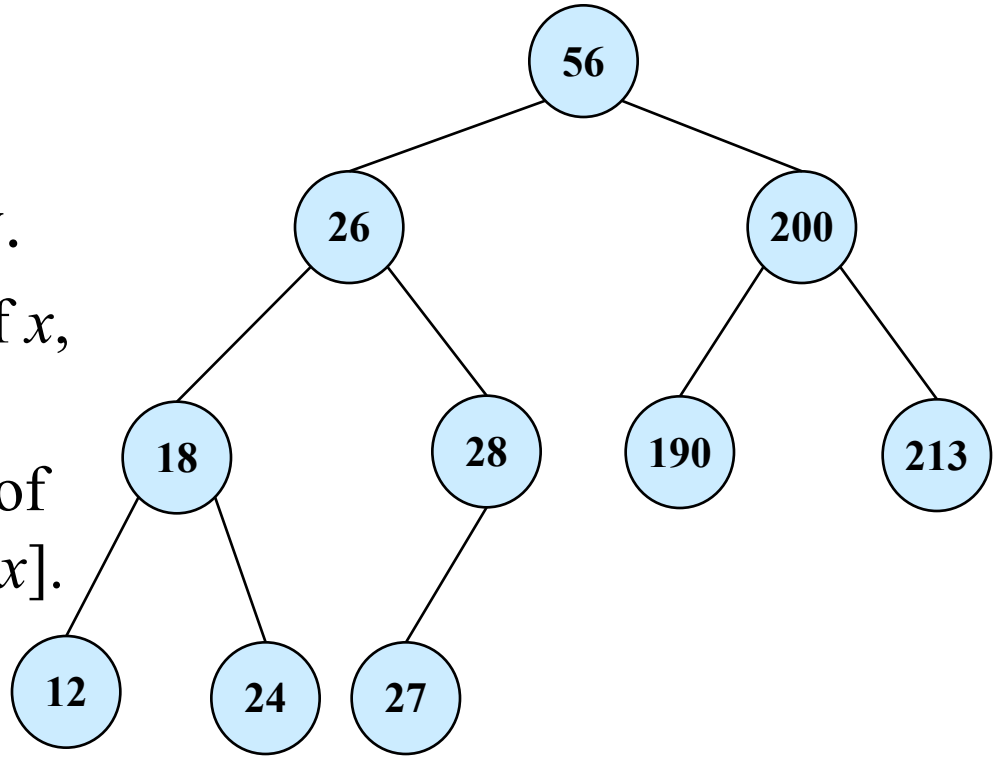
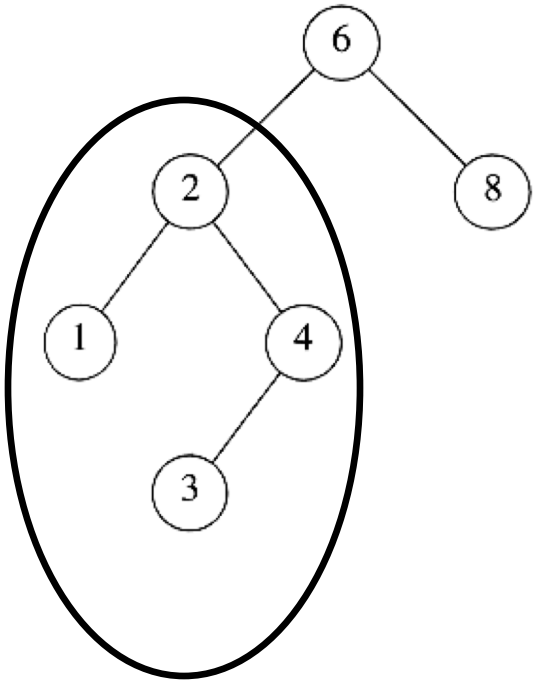
Binary Search Tree

Binary Search Tree Property

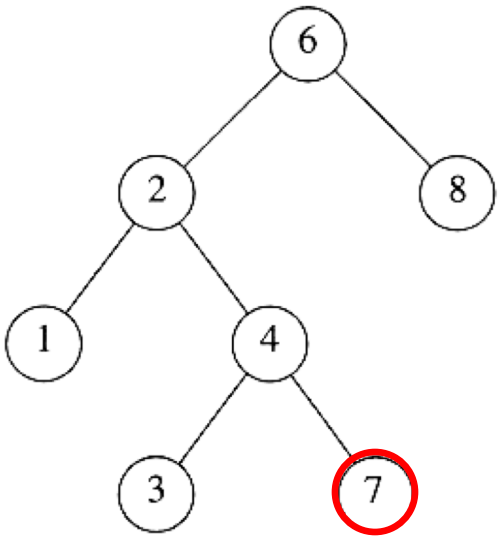
- Stored keys must satisfy the *binary search tree* property.
  - ∀ y in left subtree of x, then $key[y] \leq key[x]$.
  - ∀ y in right subtree of x, then $key[y] \geq key[x]$.



Binary Search Trees



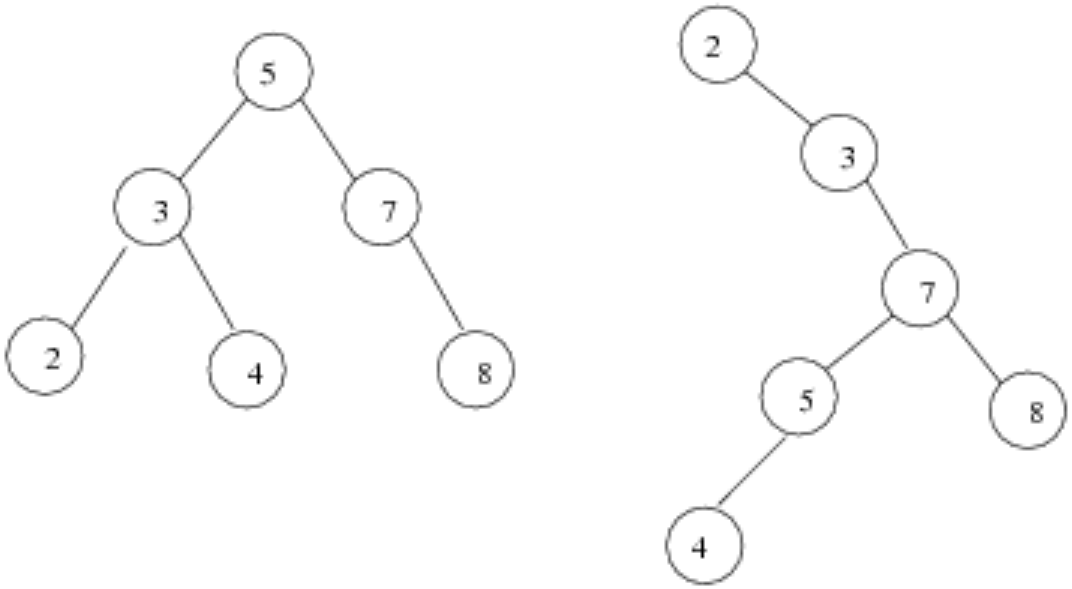
A binary search tree



Not a binary search tree

BST

Two binary search trees representing the same set:

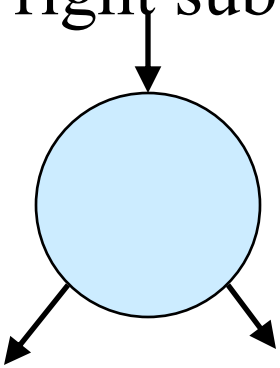


- Average height is O(log N);
- maximum height is O(N)

BST – Representation

- Represented by a linked data structure of nodes.
- *root*(*T*) points to the root of tree *T*.

- Each node contains fields:
  - *key*
  - *left* – pointer to left child: root of left subtree.
  - *right* – pointer to right child : root of right subtree.



Node Structure

```
Struct Tree
{
int data;
Struct Tree *left;
Struct Tree *right;
};
```

Binary Search Trees

- Operations On BST.
  - Search, Minimum, Maximum, Predecessor, Successor, Insert, and Delete.

- Can be used to build
  - Dictionaries.
  - Priority Queues.

Basic Operations

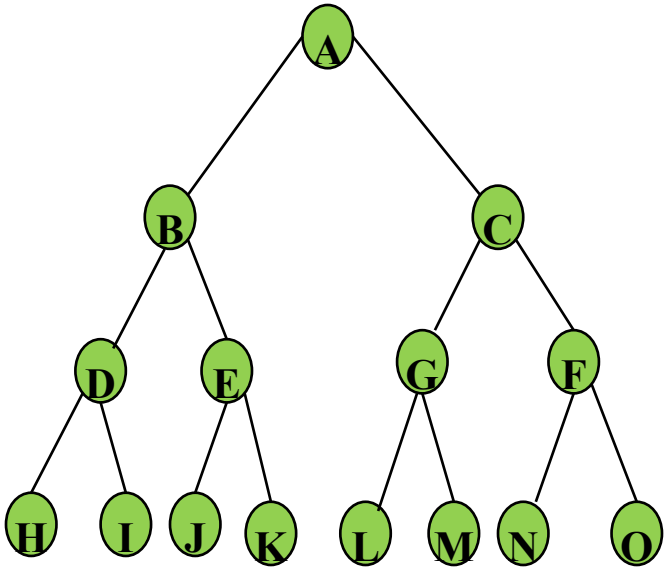
- ✓ Search - compare the values and proceed either to the left or to the right
- ✓ Insertion - unsuccessful search - insert the new node at the bottom where the search has stopped
- ✓ Deletion - replace the value in the node with the smallest value in the right subtree
  or the largest value in the left subtree.
- ✓ Retrieval in sorted order – inorder traversal

Complexity

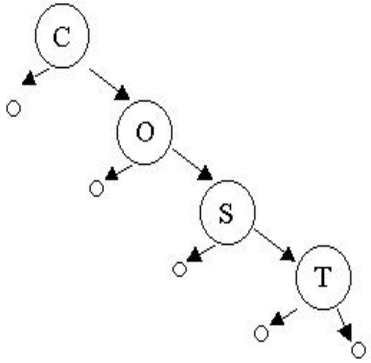
- Logarithmic, depends on the shape of the tree

  In the best case – O(log N) comparisons

  In the worst case – O(N) comparisons



Best case



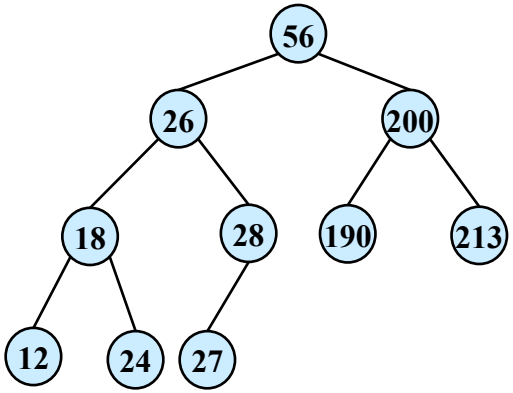
Worst case

Inorder Traversal

The binary-search-tree property allows the keys of a binary search tree to be printed, in (monotonically increasing) order, recursively.

Inorder-Tree-Walk (x)

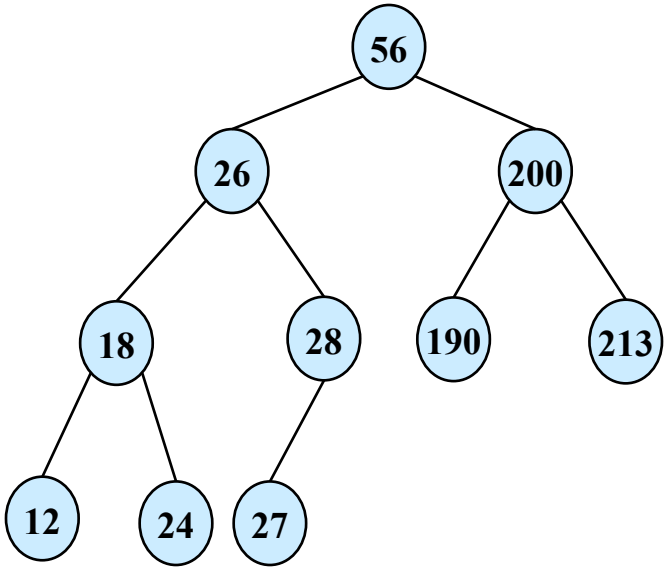
1. **if** $x \neq$ NIL
2. **then** Inorder-Tree-Walk(*left*[*p*])
3. print *key*[*x*]
4. Inorder-Tree-Walk(*right*[*p*])



- **How long does the walk take?**

Tree Search

```
struct Tree *Find(struct Tree *root, int data )
{
If(root ==Null)
    return Null;
If (data<root->data)
    return (Find ( root->left, data));
else if ( data>root->data)
        return (Find ( root->right, data));
return root;
}
```



Finding Min & Max

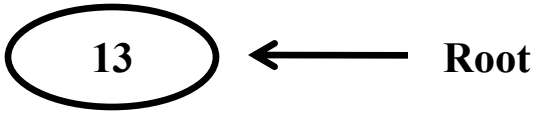
- The binary-search-tree property guarantees that:
  - » The minimum is located at the left-most node.
  - » The maximum is located at the right-most node.

```
Struct Tree *FindMin(Struct Tree *root)
{
if(root ==Null)
    return Null;
else If (root->left==Null)
        return root;
else return FindMin(root->left);
}
```

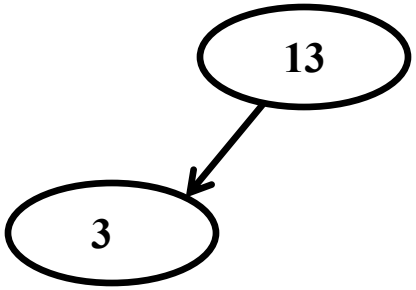
```
Struct Tree *FindMax(Struct Tree *root)
{
If(root ==Null)
    return Null;
Else If (root->right==Null)
        return root;
Else return FindMax(root->right);
}
```

Insertion in a Binary Search Tree

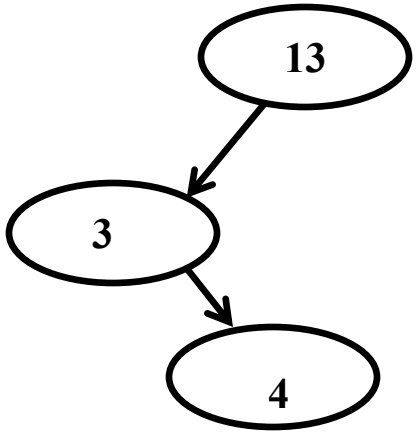
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



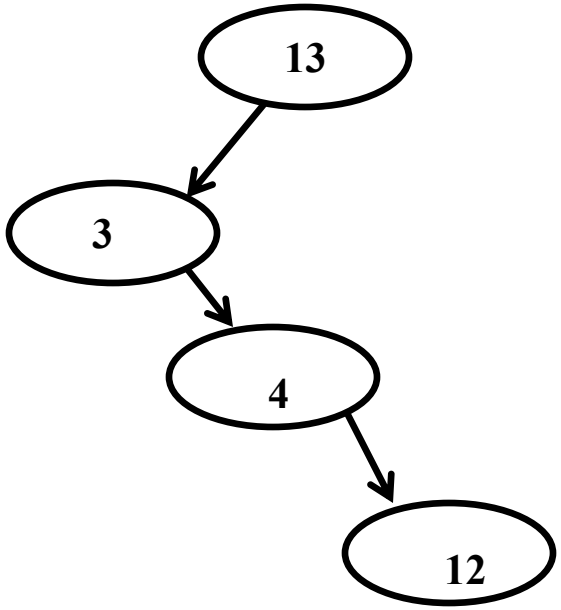
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



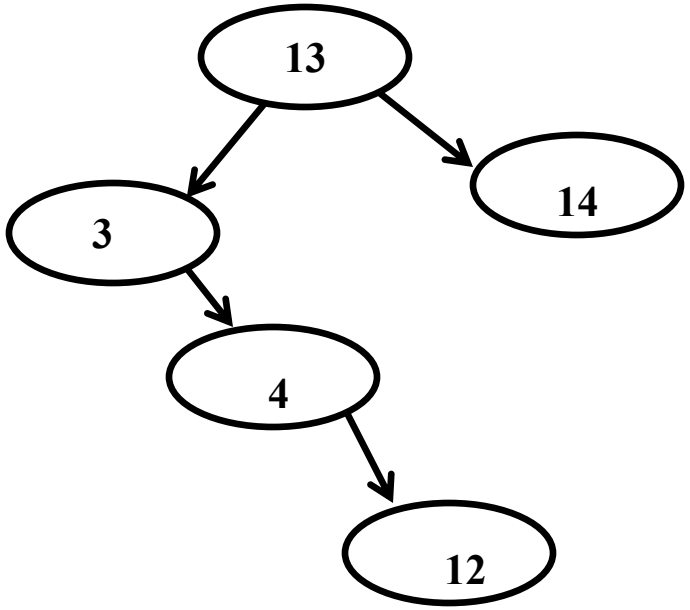
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



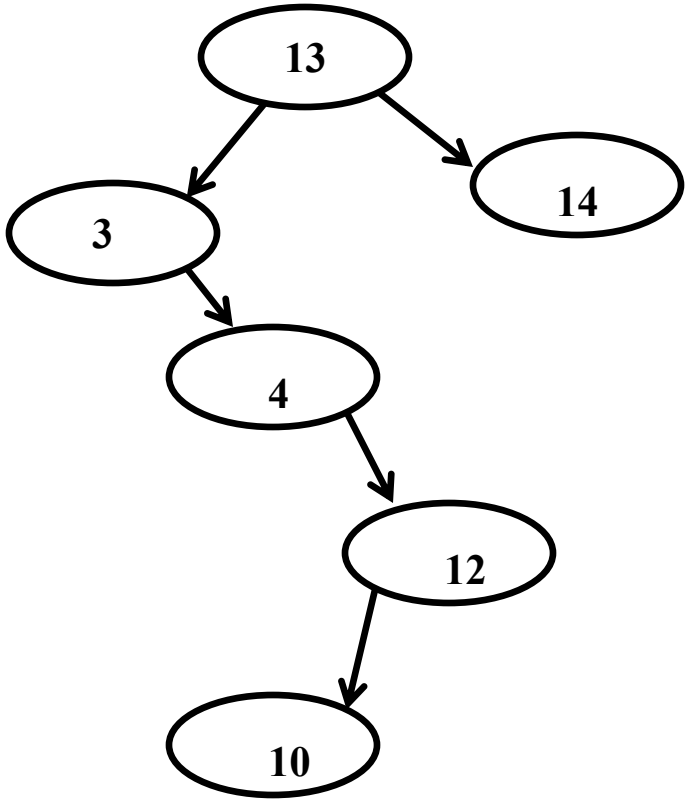
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



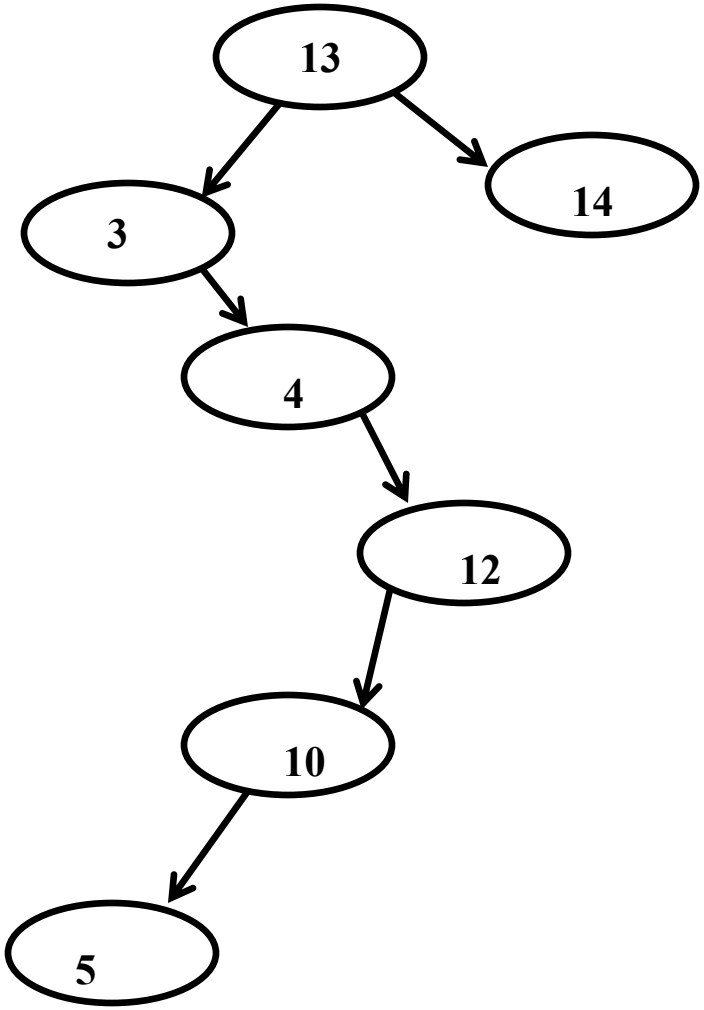
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



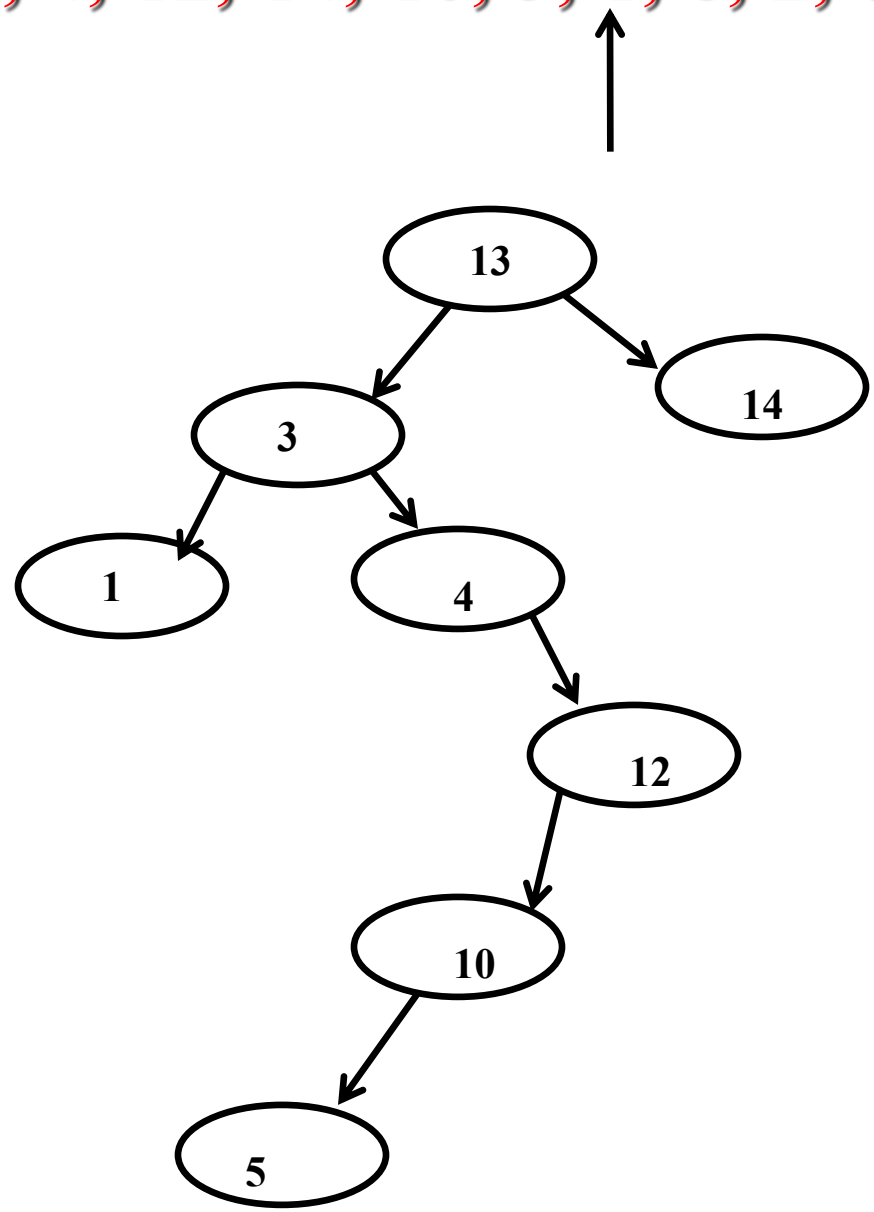
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



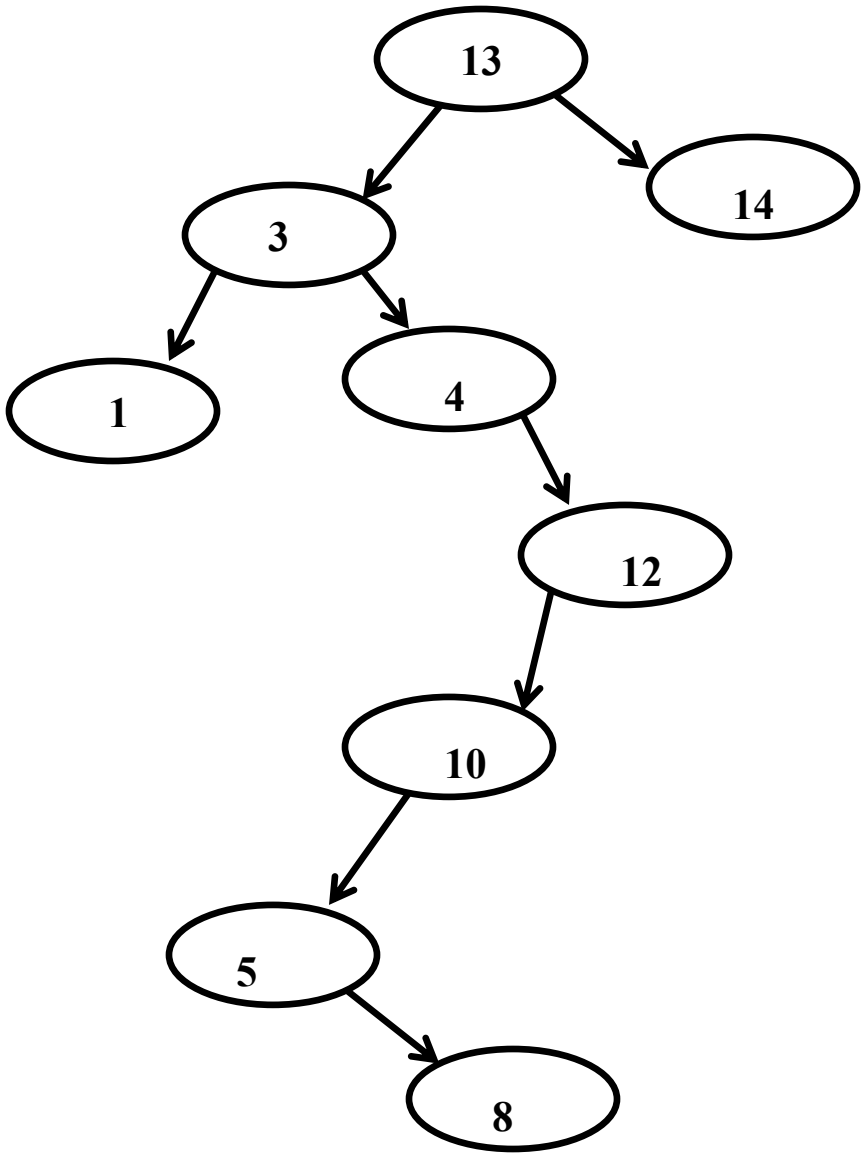
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



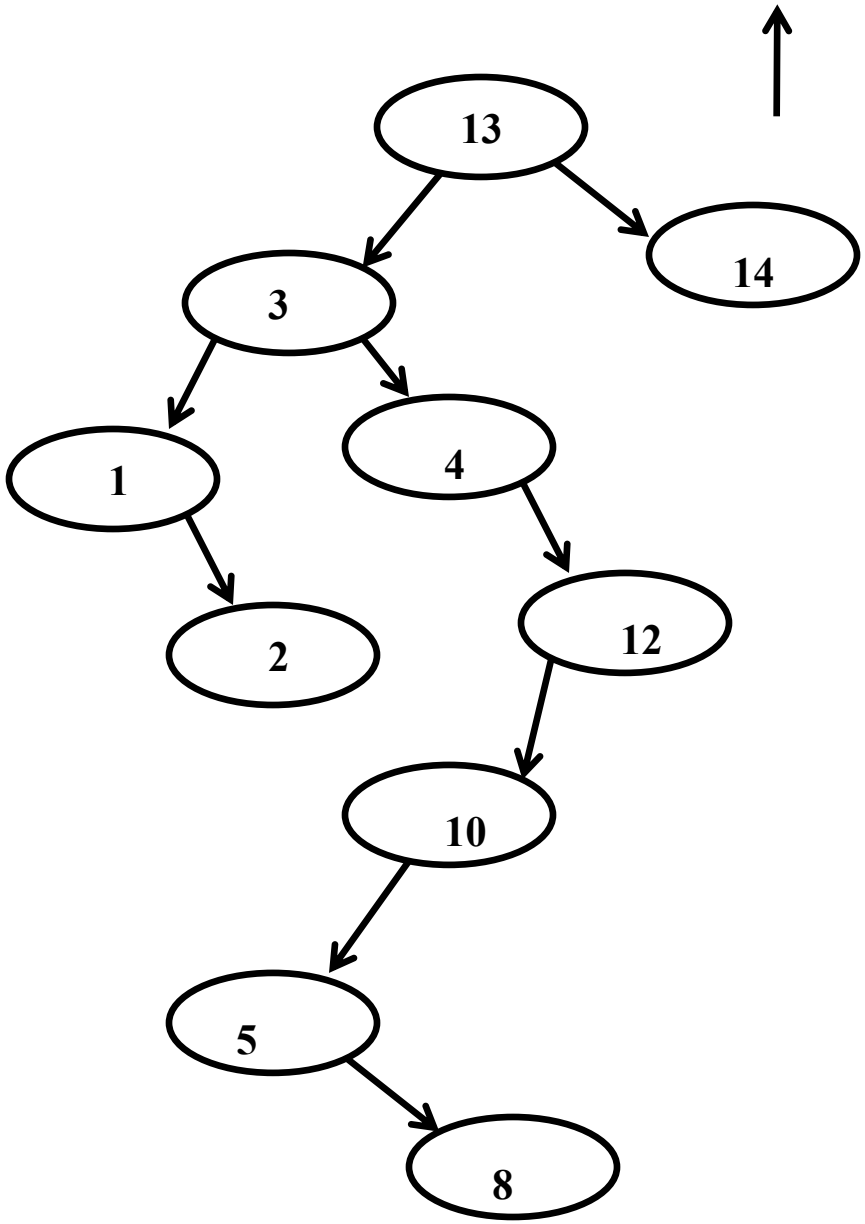
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



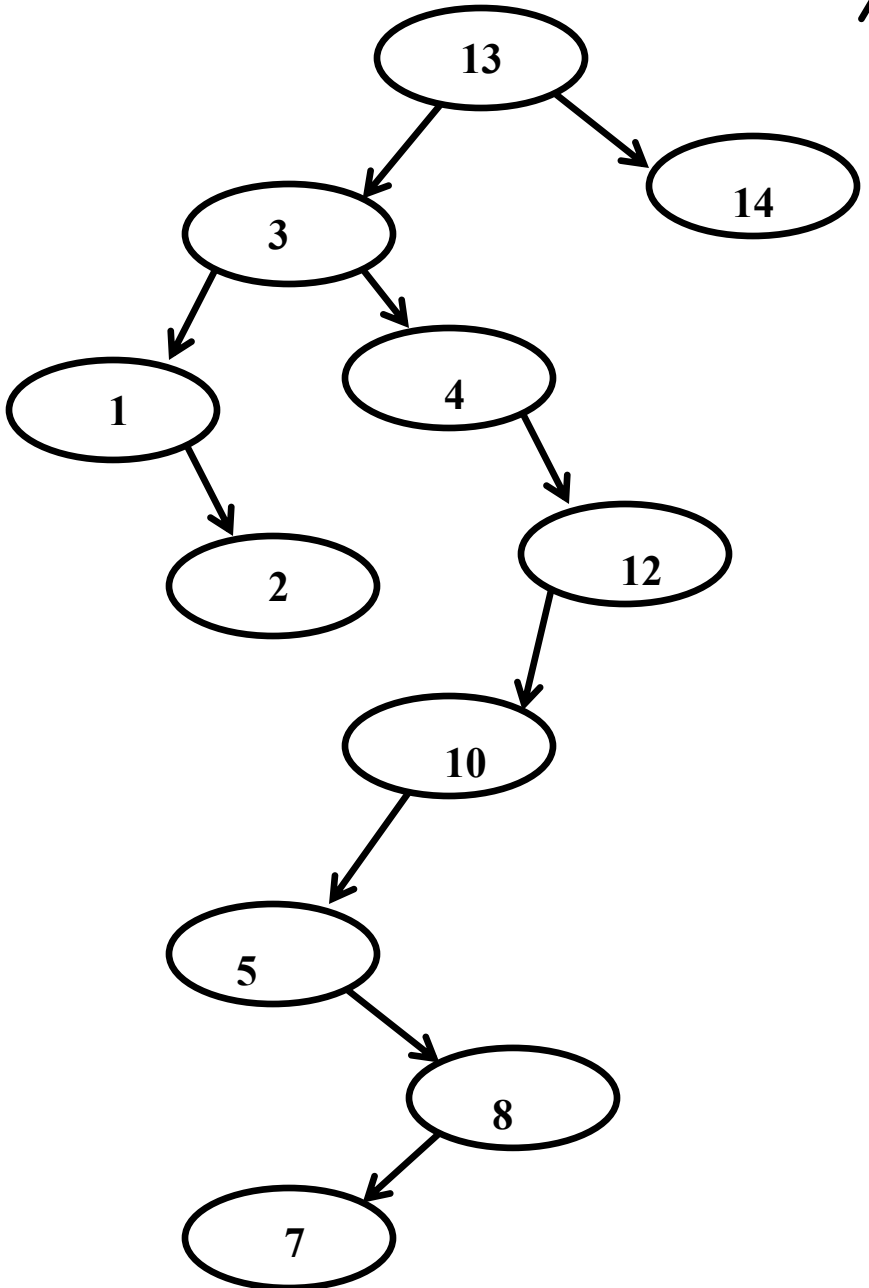
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



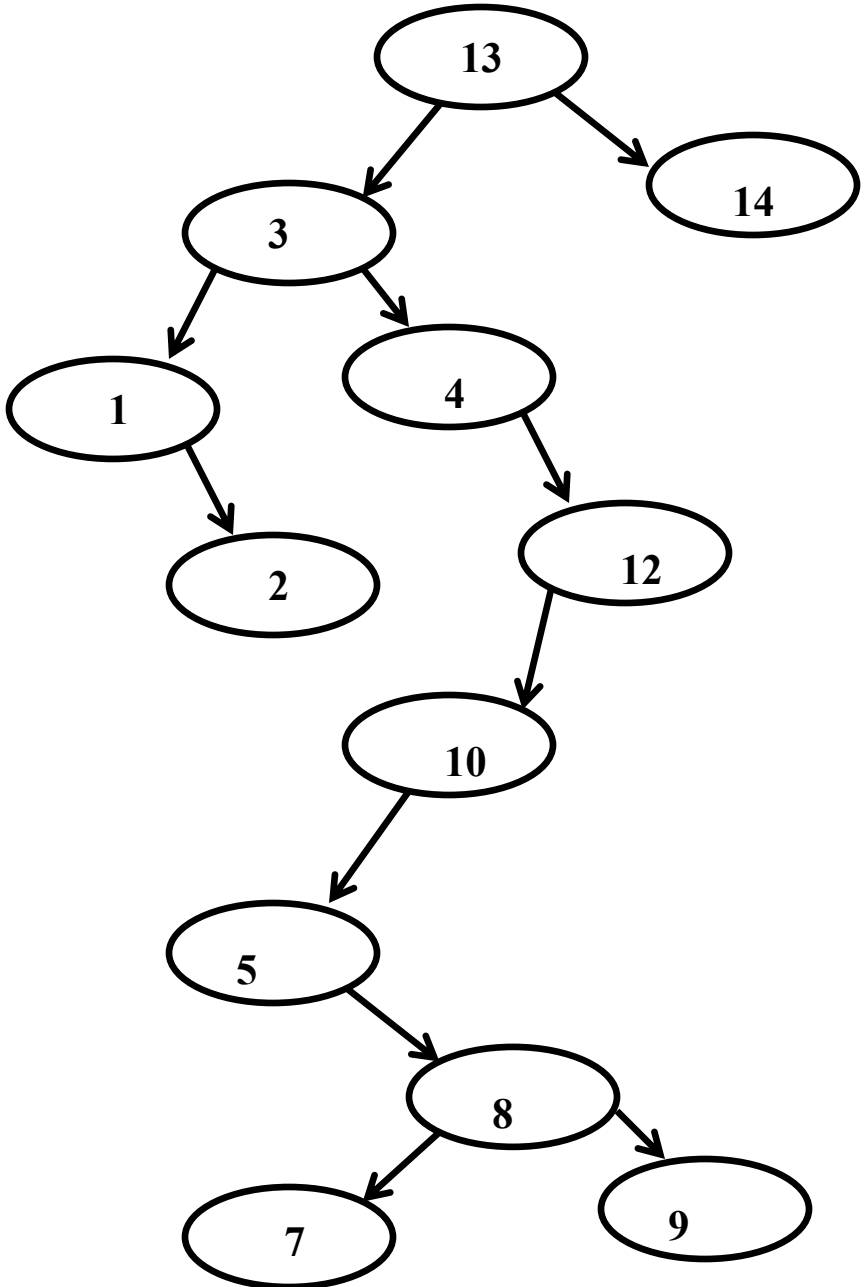
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



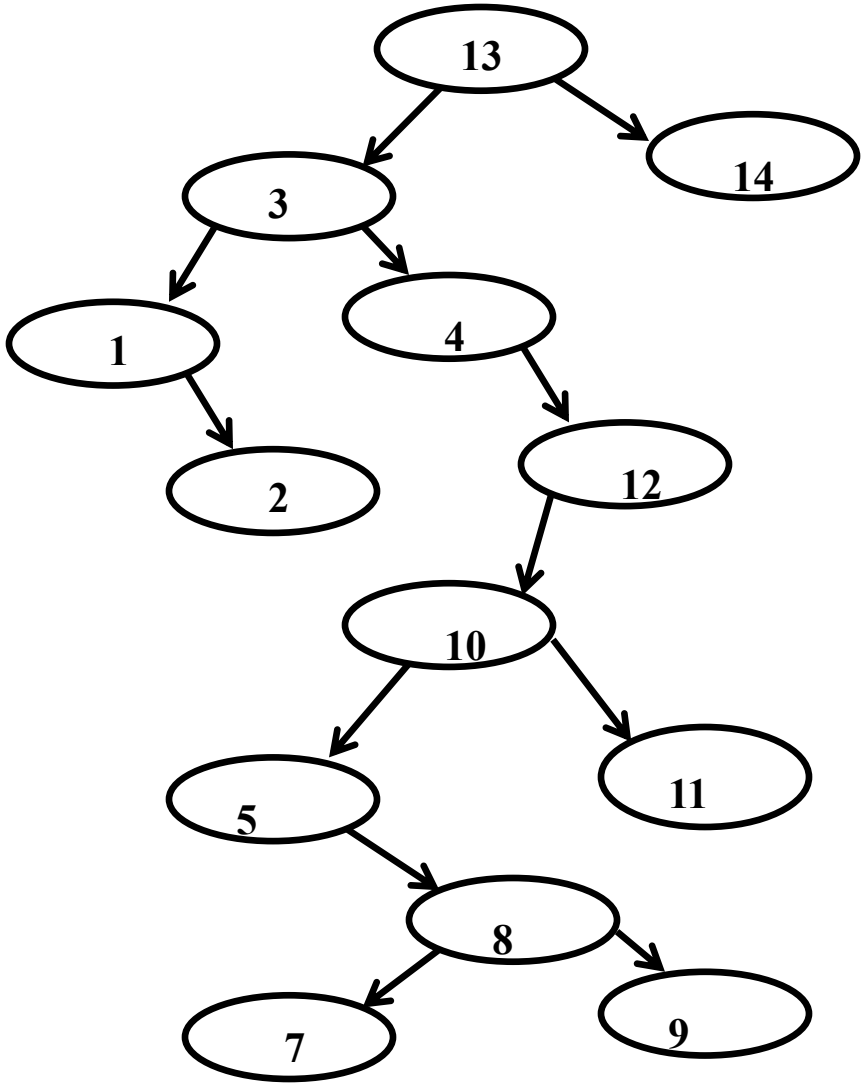
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



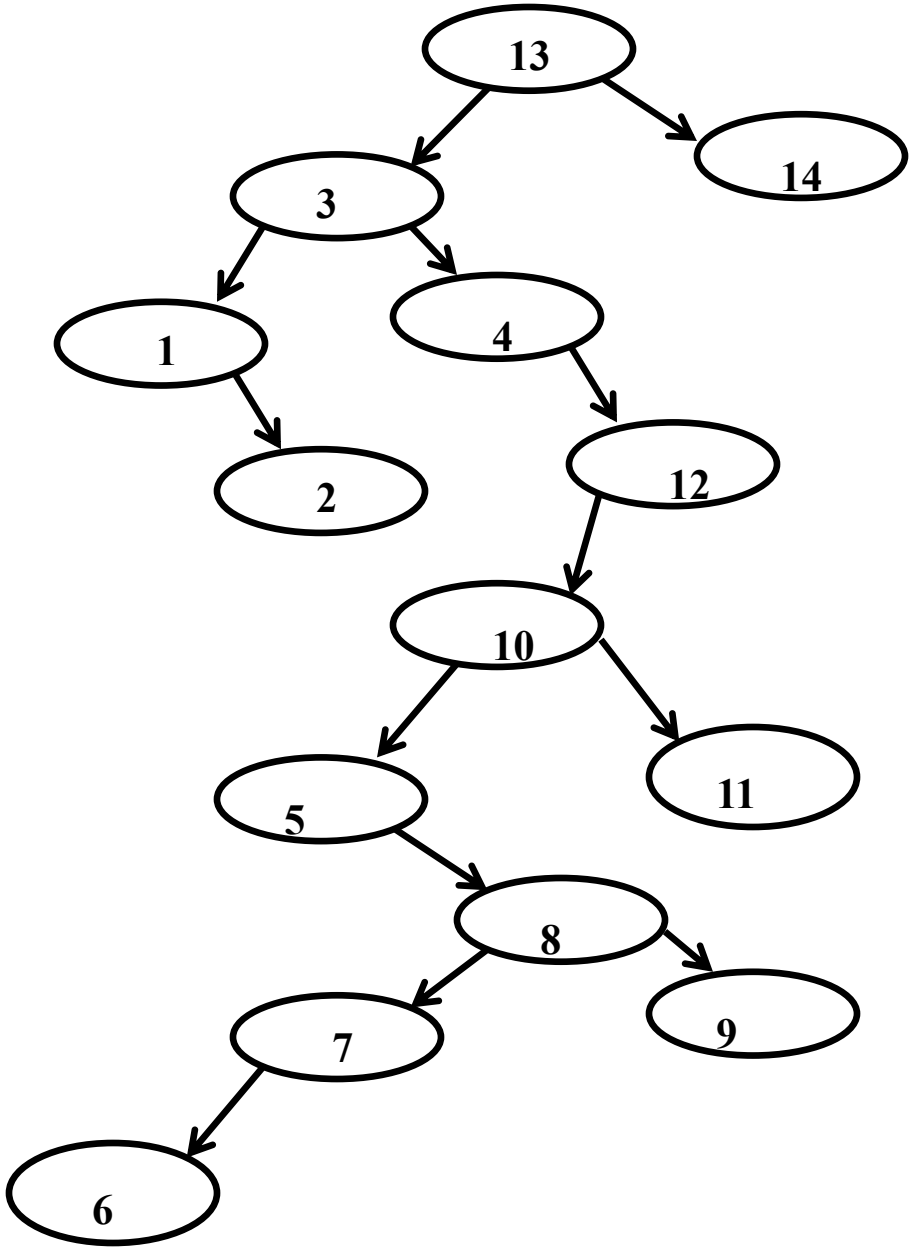
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



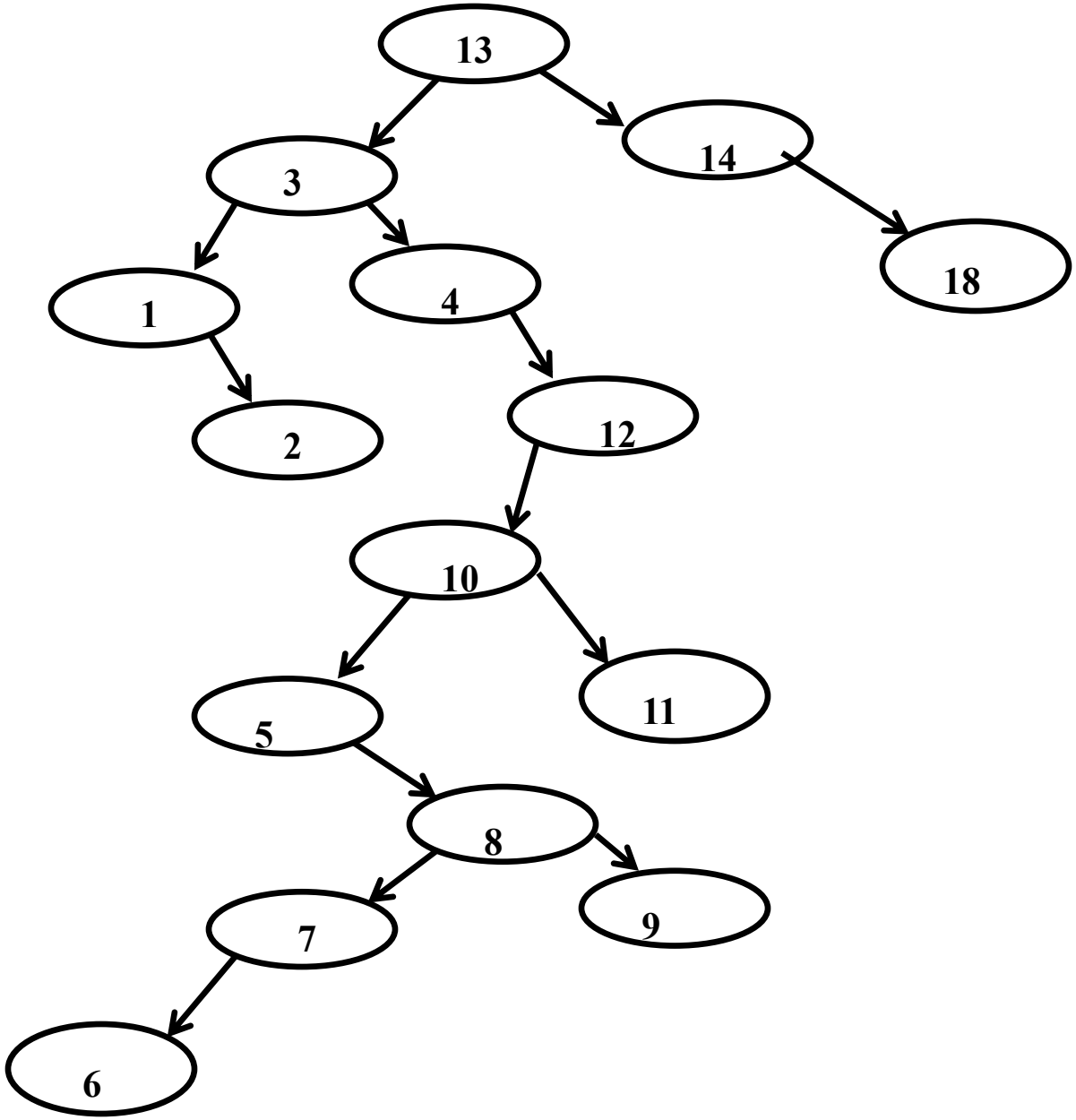
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



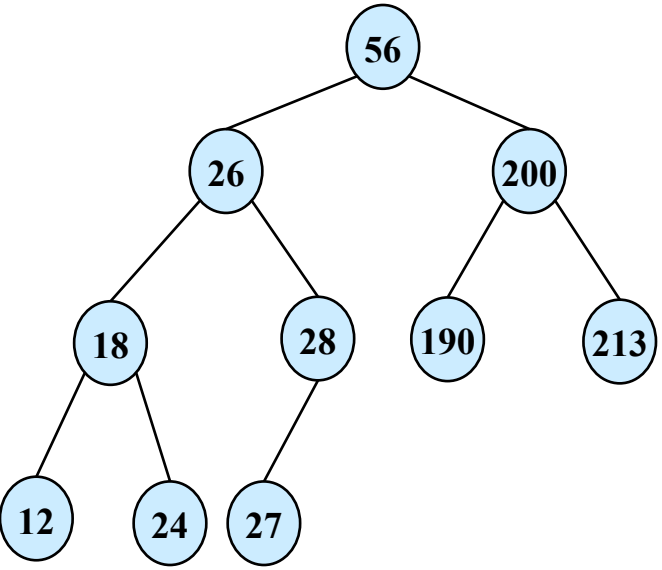
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



BST Insertion – Pseudocode




```
struct Tree *insert(struct Tree *root, int data)
{
If(root ==Null)
  {
    root=struct Tree* malloc(sizeof(struct Tree ));
    root->data=data;
    root->left=root->right=Null;
  }
else if(data<root->data)
    root->left=insert( root->left, data);
else if (data>root->data)
        root->right=insert( root->right, data);
}
Return root;
}
```

Tree-Delete (T, x)

if x has no children ♦ case 0

then remove x

if x has one child ♦ case 1

then make $p[x]$ point to child

if x has two children (subtrees) ♦ case 2

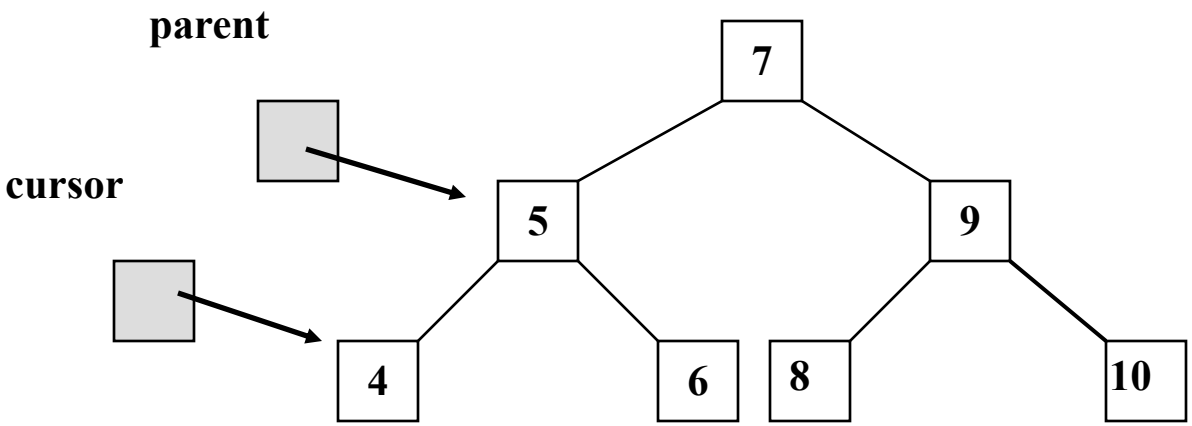
then swap x with its successor

perform case 0 or case 1 to delete it

$\Rightarrow$ TOTAL: $O(h)$ time to delete a node

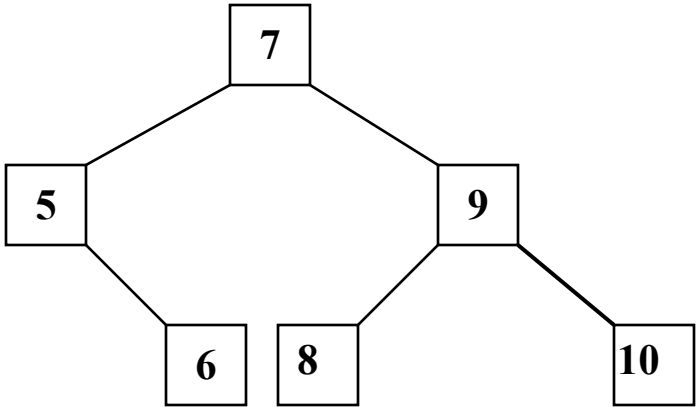
Removal in BST: Example

Case 0: **removing a node with 2 EMPTY SUBTREES**



Removing 4

replace the link in the parent with `null`

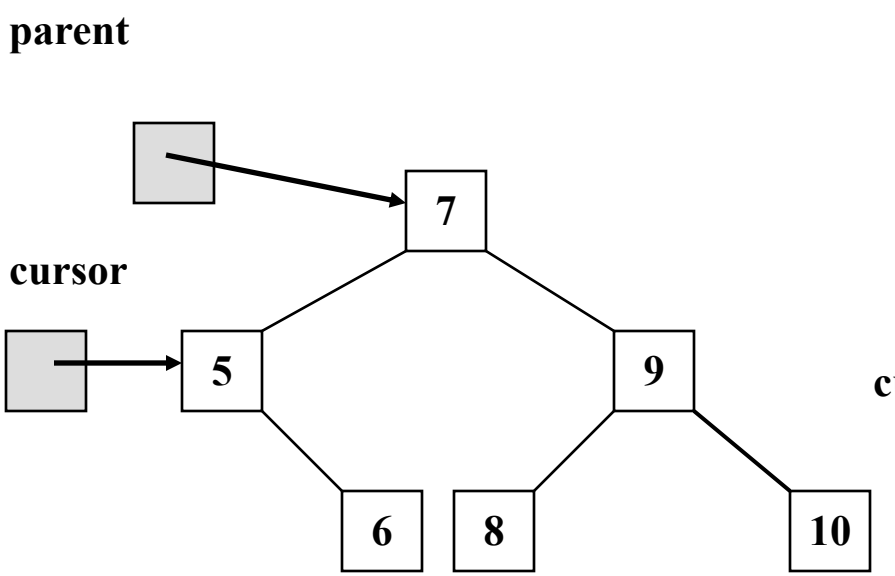


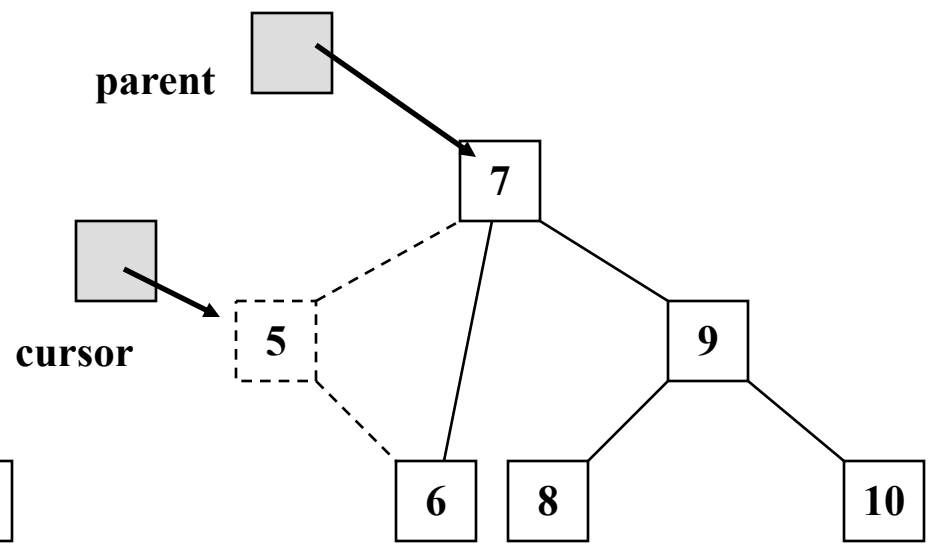
Removal in BST: Example

Case 1: **removing a node with 1 EMPTY SUBTREE**

the node has no left child:

link the parent of the node to the right (non-empty) subtree





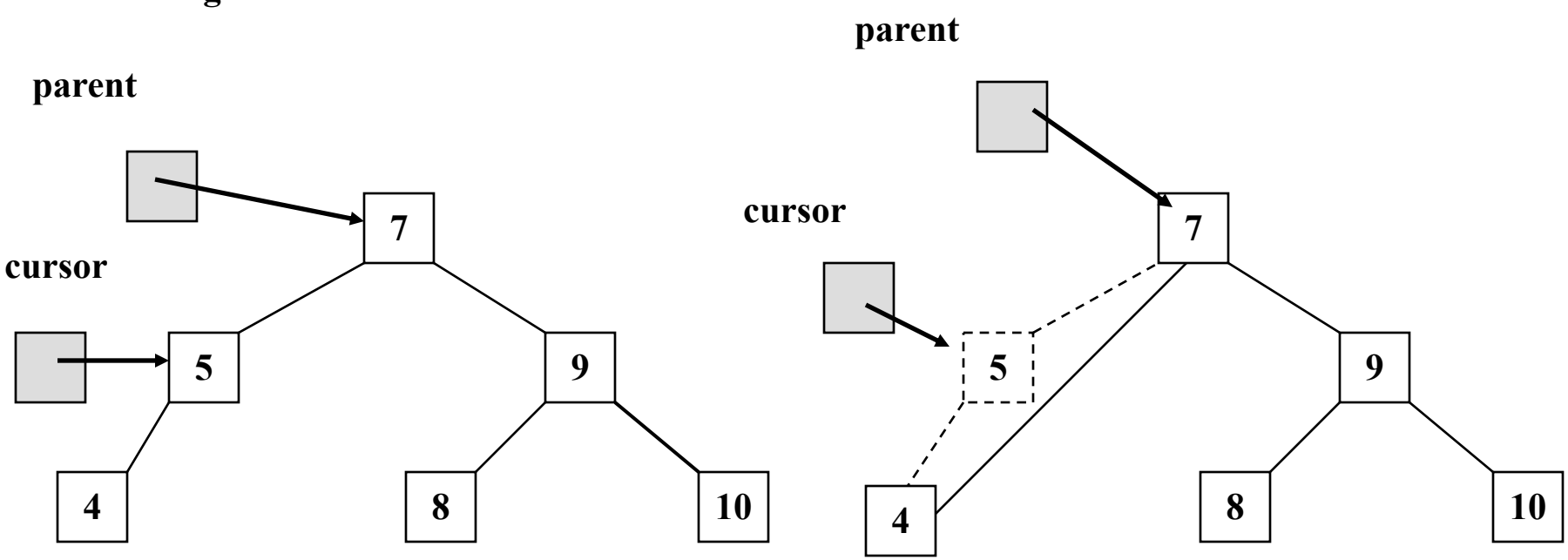
Removal in BST: Example

Case 1: removing a node with 1 EMPTY SUBTREE

the node has no right child:

link the parent of the node to the left (non-empty) subtree

Removing 5



Removal in BST: Example

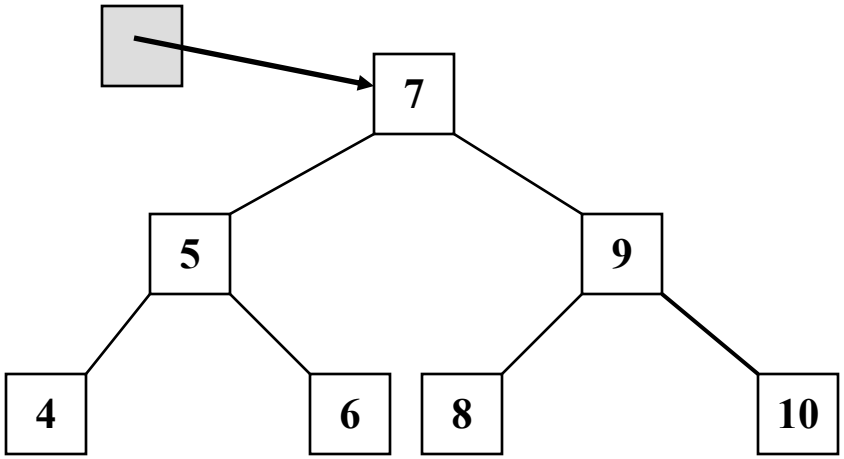
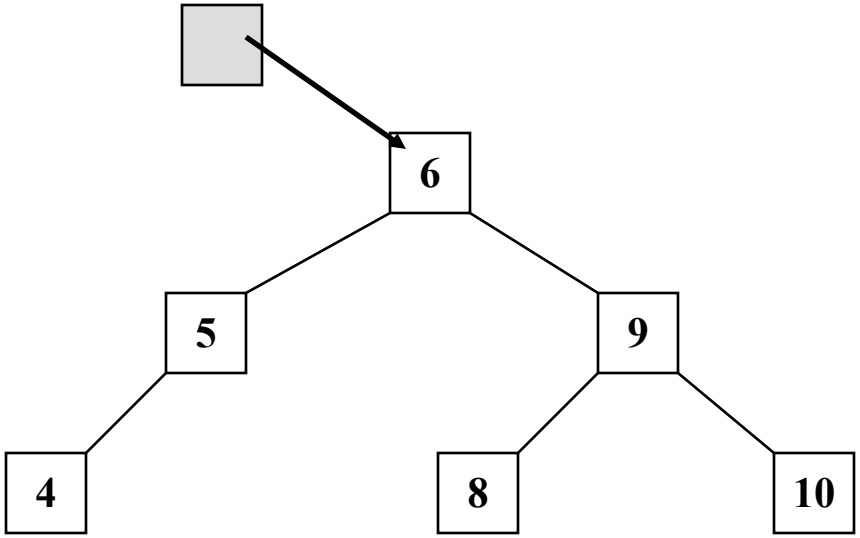
Case 2: **removing a node with 2 SUBTREES**

- **replace the node's value with the max value in the left subtree**
- **delete the max node in the left subtree**

What other element can be used as replacement?

Removing 7

cursor



cursor